

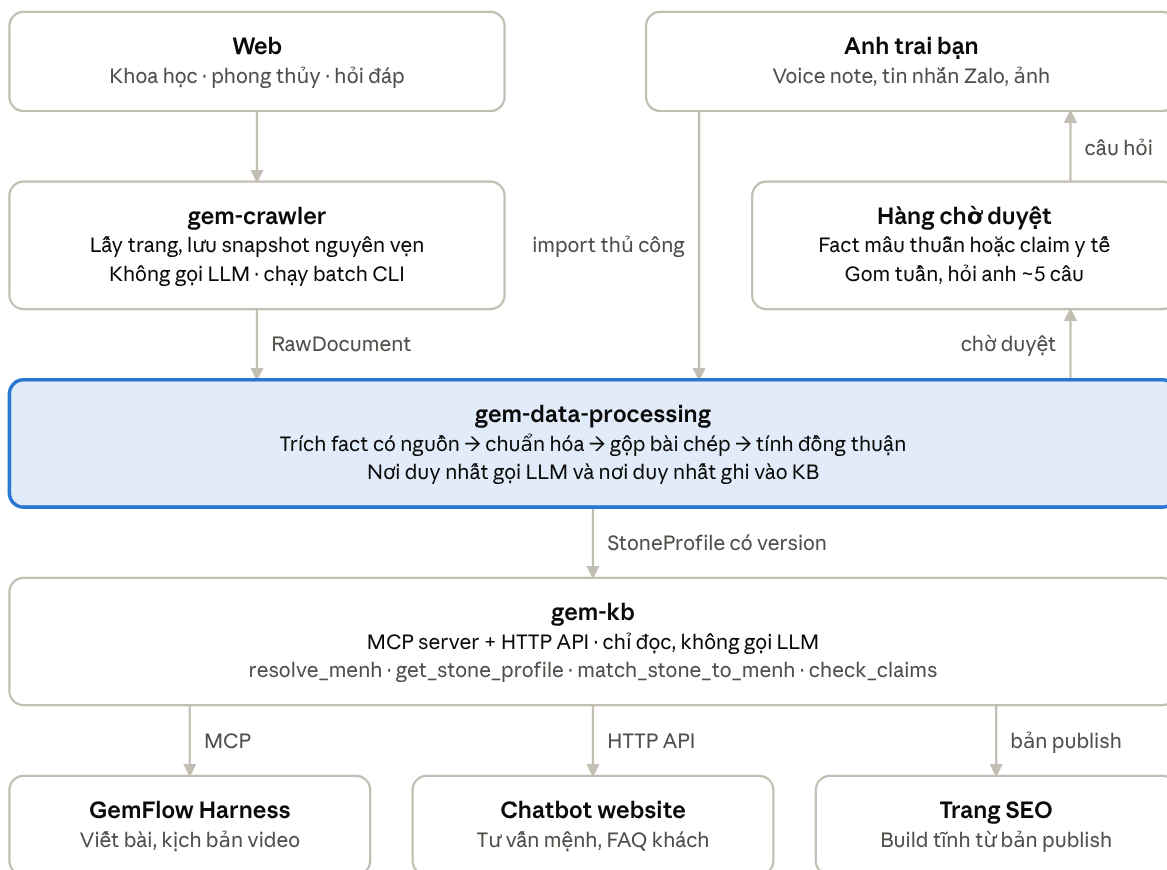
OmniGem Knowledge Platform — High-Level Design

Oct 1, 2026 · @Leonard

Tri thức về đá và phong thủy đi một chiều qua 3 project: gem-crawler lấy dữ liệu thô, gem-data-processing tinh chế thành hồ sơ đá đã kiểm chứng, gem-kb phục vụ hồ sơ đó cho website, chatbot và các agent viết bài. Chỉ data-processing được ghi vào KB.

Tổng quan

Tri thức đi một chiều: thô → tinh → phục vụ



Giá, tồn kho, SKU nằm ở Sanity CMS của website; chatbot lấy qua search_products.

Kiến trúc · 3 project, 2 nguồn vào, 3 nơi dùng

Mọi nguồn thô đổ vào cùng một xưởng tinh chế; chỉ fact đã qua đồng thuận hoặc đã duyệt mới được publish sang KB, và mọi kênh đọc từ KB.

Nguyên tắc thiết kế

Năm quy tắc này giữ cho ranh giới giữa 3 project luôn sạch.

1. **Dữ liệu chảy một chiều:** crawler → data-processing → kb. Không project nào ghi ngược lên tầng trước.
2. **Chỉ data-processing được ghi vào KB.** Agent, chatbot hay website chỉ đọc.
3. **Giữ data thô, không xóa.** Sửa logic tinh chế thì chỉ chạy lại data-processing, không phải crawl lại.
4. **Mỗi fact có nguồn.** Mọi giá trị trong hồ sơ tinh truy ngược được tới URL hoặc tin nhắn gốc và câu làm bằng chứng.
5. **Luật tính toán là code, không phải văn bản.** Ngữ hành, tương sinh/khắc, Can Chi do code tất định xử lý; LLM chỉ trích xuất và diễn đạt.

Giá, tồn kho và SKU không thuộc hệ thống này. Chúng ở lại Sanity CMS của website; KB chỉ tham chiếu loại đá.

gem-crawler

Crawler đi lấy trang web và lưu nguyên vẹn, không hiểu nội dung. Nó chạy batch trên máy của bạn bằng lệnh CLI, không phải server.

Đầu vào	File config: seed list tên đá + tên gọi khác, danh sách nguồn theo tầng tin cậy, từ khóa tìm kiếm
Đầu ra	<code>RawDocument</code> : snapshot HTML + text đã làm sạch, url, ngày lấy, tầng nguồn, đá mục tiêu
Lưu trữ	File JSON hoặc SQLite cục bộ, chia theo lần chạy
Stack	Node/TypeScript, <code>crawlee</code> (Playwright, hàng đợi, retry, giới hạn tốc độ)

Thành phần chính

- **Source registry:** danh sách nguồn, mỗi nguồn gắn tầng: tầng 1 khoa học (Wikipedia, Mindat, bách khoa đá quý của viện kiểm định), tầng 2 phong thủy tiếng Việt, tầng 3 hỏi đáp.
- **Fetcher:** tôn trọng `robots.txt`, giới hạn tốc độ theo domain, bỏ qua trang đã lấy nếu chưa hết hạn.

- **Raw store:** lưu snapshot kèm hash nội dung để phát hiện trang đã đổi.

Không làm: không crawl sàn TMĐT (Shopee, TikTok Shop, Lazada) hay group Facebook, vì điều khoản cấm và có dữ liệu cá nhân. Không chạy LLM.

gem-data-processing

Data-processing biến mọi nguồn thô thành các fact nhỏ có nguồn, rồi tổng hợp fact đã chấp nhận thành hồ sơ đá tinh. Đây là nơi duy nhất gọi LLM và nơi duy nhất ghi vào KB.

Hai loại đầu vào, cùng một xưởng

- `RawDocument` từ gem-crawler.
- Tri thức từ anh trai bạn: voice note, tin nhắn Zalo, ảnh kèm chú thích, đưa vào bằng lệnh import thủ công.

Pipeline

1. **Chuyển thành text:** voice → transcript, ảnh → mô tả; trang web đã có text sẵn.
2. **Trích fact:** LLM tách text thành `Fact` theo schema zod, mỗi fact kèm câu gốc làm bằng chứng.
3. **Chuẩn hóa:** "đá hồng", "rose quartz" → `thach-anh-hong`; giá trị ép về đúng enum (hành, màu, mức độ claim).
4. **Gộp bài chép:** so độ trùng văn bản giữa các trang; trang gần giống nhau chỉ tính là một nguồn.
5. **Đôi chiếu:** gom fact theo cặp (đá, trường), tính mức tin cậy, kiểm tra với luật ngũ hành trong code.
6. **Phân loại:** tự chấp nhận, hoặc đưa vào hàng chờ hỏi anh trai bạn (gửi gộp mỗi tuần, chỉ đá đang có hàng).
7. **Publish:** ghi `StoneProfile` có version vào KB, kèm file JSON Schema của phiên bản đó.

Mức tin cậy của mỗi fact

Mức	Điều kiện	Agent được dùng thể nào
Đã xác minh	Nguồn tầng 1, hoặc anh trai bạn xác nhận	Khẳng định
Đồng thuận	≥ 3 nguồn độc lập trùng nhau, không nguồn nào phản bác	Dùng bình thường

Mức	Điều kiện	Agent được dùng thể nào
Một nguồn	1-2 nguồn	Giọng mềm ("nhiều người tin rằng...") hoặc bỏ qua
Mâu thuẫn	Các nguồn nói khác nhau, hoặc trái luật ngũ hành	Không dùng; vào hàng chờ duyệt

Trường khoa học (độ cứng, thành phần, xử lý nhân tạo) chỉ nhận từ nguồn tầng 1. Fact có dấu hiệu claim y tế luôn vào hàng chờ, không bao giờ tự chấp nhận. Hàng chờ ban đầu chỉ cần là một bảng tính.

gem-kb

KB phục vụ hồ sơ đá đã publish, chỉ đọc, không gọi LLM. Một core, hai adapter: MCP server cho agent và Claude, HTTP API cho website và chatbot.

Tool theo ý định

Tool	Trả về
<code>resolve_menh(birthYear, birthDate?)</code>	Can Chi, nạp âm, hành bản mệnh (code tắt định)
<code>get_stone_profile(stone)</code>	Hồ sơ đầy đủ; nhận cả tên gọi khác
<code>match_stone_to_menh(stone, menh)</code>	Quan hệ sinh/khắc, điểm, ý chính để viết
<code>list_stones_for_menh(menh, colorFamily?)</code>	Danh sách đá hợp mệnh
<code>get_authenticity_guide(stone)</code>	Cách phân biệt thật/giả
<code>get_care_guide(stone)</code>	Cách bảo quản
<code>check_claims(text)</code>	Claim vi phạm và cách diễn đạt an toàn thay thế

Mỗi kết quả trả kèm mức tin cậy và nguồn của từng trường, để agent chọn giọng khẳng định hay giọng mềm.

Ai dùng và dùng thể nào

- **GemFlow Harness** (viết bài, kịch bản video): qua MCP, thay cho bảng dữ liệu cứng trong `src/domain/data/`.
- **Chatbot website** (`/api/chat` trong `gem-digital-center-hub`): qua HTTP API cho tri thức đá và mệnh; `search_products` vẫn gọi Sanity cho hàng và giá. Cần thêm một ADR mới trong project website cho phần này.
- **Trang SEO** ("Thạch anh hồng hợp mệnh gì"): website build tĩnh từ bản publish của KB.

Chatbot dùng một view hẹp hơn: chỉ fact mức "Đã xác minh" và "Đồng thuận", vì nó trả lời khách trực tiếp mà không qua người duyệt.

Hợp đồng dữ liệu

Vì 3 project tách repo và không có package dùng chung, mỗi ranh giới do project phía trước sở hữu và xuất ra dưới dạng JSON Schema có version.

Ranh giới	Hợp đồng	Chủ sở hữu	Bên nhận kiểm tra bằng
crawler → data-processing	<code>RawDocument</code>	gem-crawler	JSON Schema đi kèm mỗi lần chạy crawl
data-processing (nội bộ)	<code>Fact</code>	gem-data-processing	zod
data-processing → kb	<code>StoneProfile</code> + <code>release.json</code> (version, ngày, số đá)	gem-data-processing	JSON Schema đi kèm mỗi bản publish
kb → website, harness	Tool MCP + HTTP API	gem-kb	Mô tả tool và OpenAPI

Các trường chính của `Fact` : đá, trường, giá trị, nguồn (url hoặc tin nhắn), tăng nguồn, câu gốc làm bằng chứng, mức tin cậy, trạng thái duyệt.

Các nhóm trường của `StoneProfile` : định danh (id, tên, tên gọi khác), khoa học (thành phần, độ cứng, xử lý nhân tạo), phong thủy (hành, màu, mệnh hợp/ky), ý nghĩa dân gian kèm nhãn mức độ claim, nhận biết thật/giả, bảo quản, FAQ, ghi chú hình ảnh cho video, độ đầy đủ, nguồn. Danh sách trường chi tiết sẽ chốt ở giai đoạn P0.

Lộ trình

P0-P3 xong trong khoảng 2 tuần, mỗi bước có tiêu chí dừng

Thứ tự triển khai 3 project; ô thoi = điều kiện để sang bước sau



Lộ trình · 5 giai đoạn, 4 cổng

Bước nào cũng bắt đầu từ các loại đá đang có hàng ở cửa hàng, rồi mới mở rộng ra cả seed list.

Câu hỏi còn mở

- ☐ Chọn hệ quy chiếu phong thủy nào làm chuẩn khi các nguồn mâu thuẫn? Mặc định: cách anh trai bạn đang tư vấn khách.
- ☐ Danh sách loại đá cửa hàng đang có hàng, để ưu tiên crawl và duyệt trước.
- ☐ KB chạy ở đâu: cùng Vercel với website, hay một server riêng?
- ☐ Ngưỡng "Đồng thuận" là 3 nguồn có đủ không, hay cần cao hơn cho trường phong thủy?
- ☐ Ai dành thời gian duyệt hàng chờ mỗi tuần, và bao nhiêu phút?